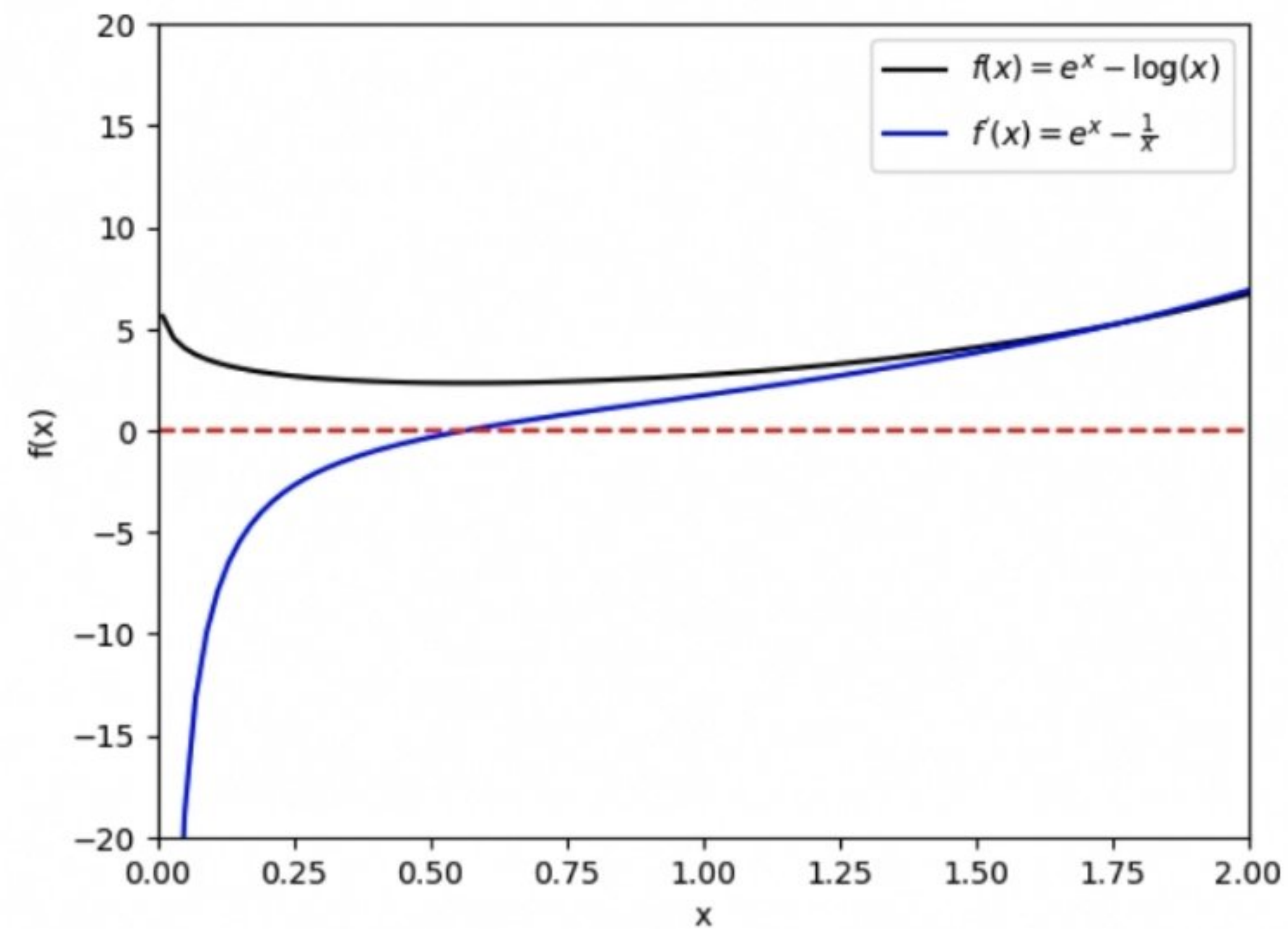


• Descenso del Gradiente

plt.show()



Podemos implementar el método con la siguiente función

```
[4]: def gradient_descent(df dx, x, learning_rate = 0.1, num_iterations = 100):  
    for iteration in range(num_iterations):  
        x = x - learning_rate * df dx(x)  
  
        #if iteration >= (num_iterations - 5):  
        #    print(f"iteración {iteration}, x:{x}")  
    return x
```

En esta función de gradiente descendiente tenemos tres parámetros: `num_iterations`, `learning_rate`, `x` (punto inicial).

- Regression Lineal

$$\beta = (X^T X)^{-1} X^T Y$$

Sol. analítico

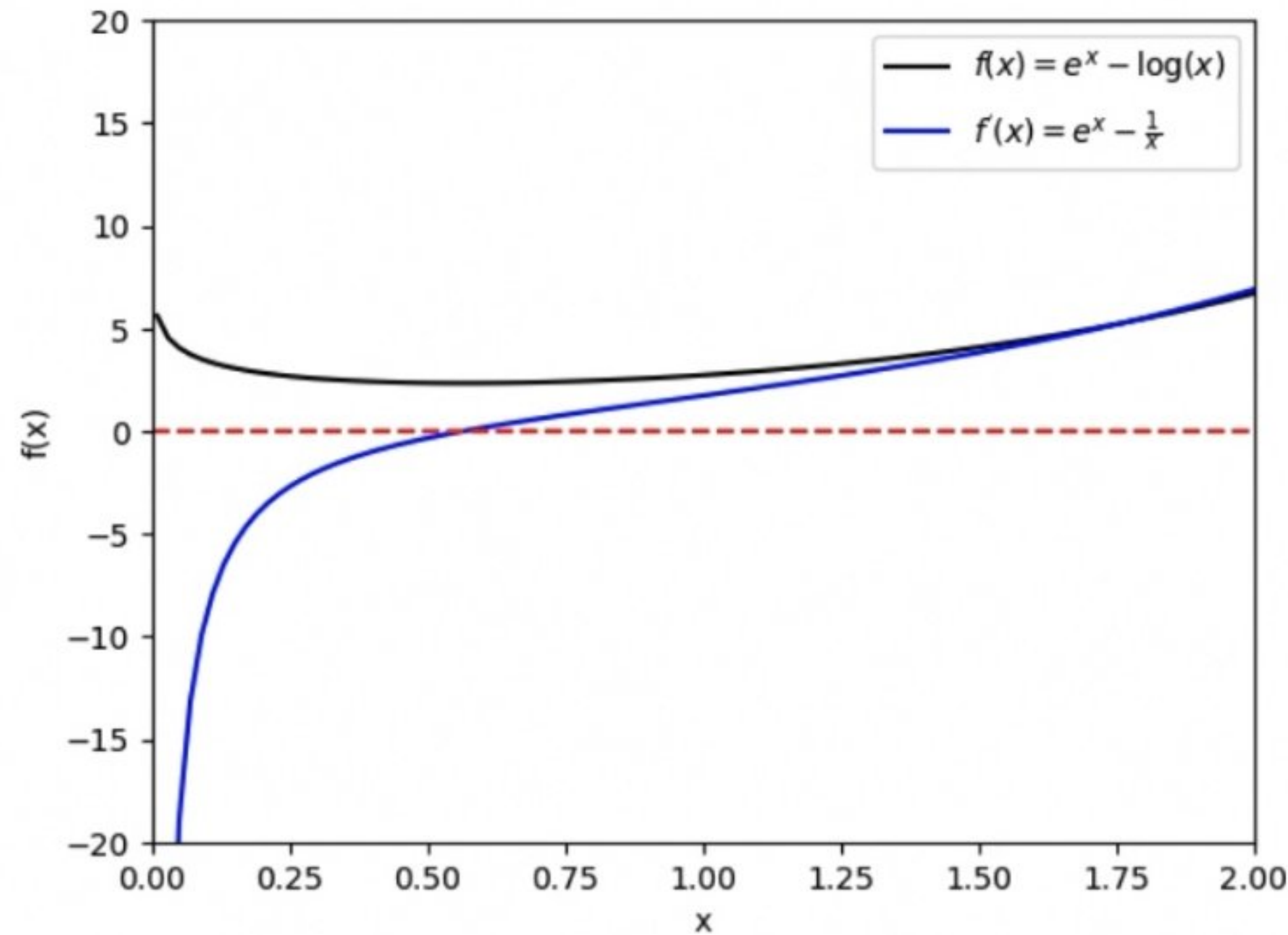
operador
nabla

$$\beta_{k+1} = \beta_k - \alpha \nabla \alpha$$

$$J(\beta)$$

$$\nabla = \left(\frac{\partial}{\partial \beta_0}, \frac{\partial}{\partial \beta_1}, \dots \right)$$

plt.show()



$$f(x) = e^x - \log(x)$$

$$\frac{df(x)}{dx} = e^x - \frac{1}{x}$$

$$\text{El punto m.m.m. } \frac{df(x)}{dx} = 0$$

$$e^x - \frac{1}{x} = 0 \Rightarrow e^x = \frac{1}{x}$$

Podemos implementar el método con la siguiente función

```
[4]: def gradient_descent(df dx, x, learning_rate = 0.1, num_iterations = 100):  
    for iteration in range(num_iterations):  
        x = x - learning_rate * df dx(x)  
  
        #if iteration >= (num_iterations - 5):  
        #    print(f"iteración {iteration}, x:{x}")  
    return x
```

En esta función de gradiente descendiente tenemos tres parámetros: `num_iterations`, `learning_rate`, `x` (punto inicial).

$$e^x = \frac{1}{x}$$

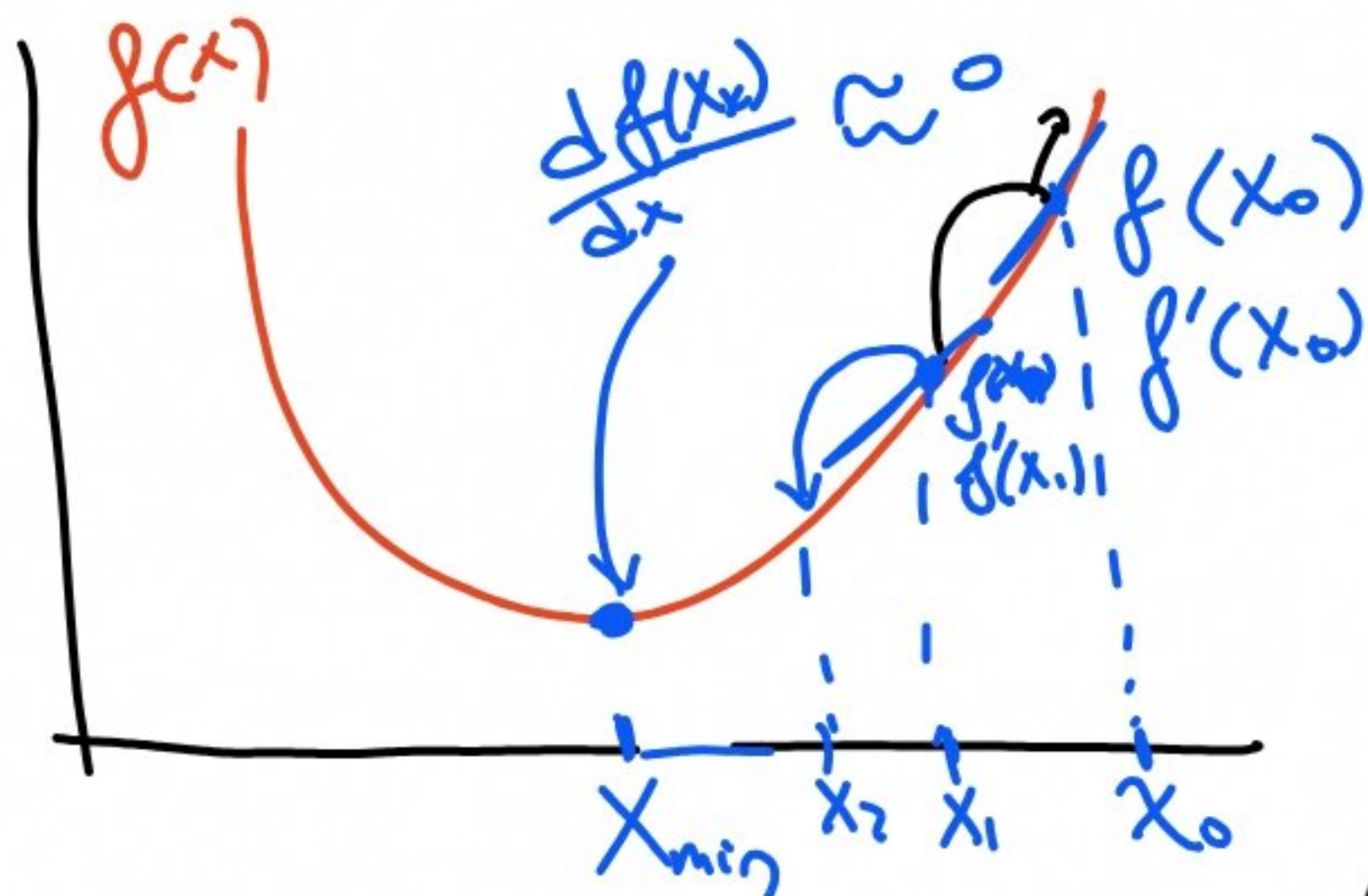
$$\Rightarrow xe^x = 1$$

$$\Rightarrow \ln(xe^x) = \ln(1) = 0$$

$$\ln(x) + \underbrace{\ln(e^x)}_x = 0$$

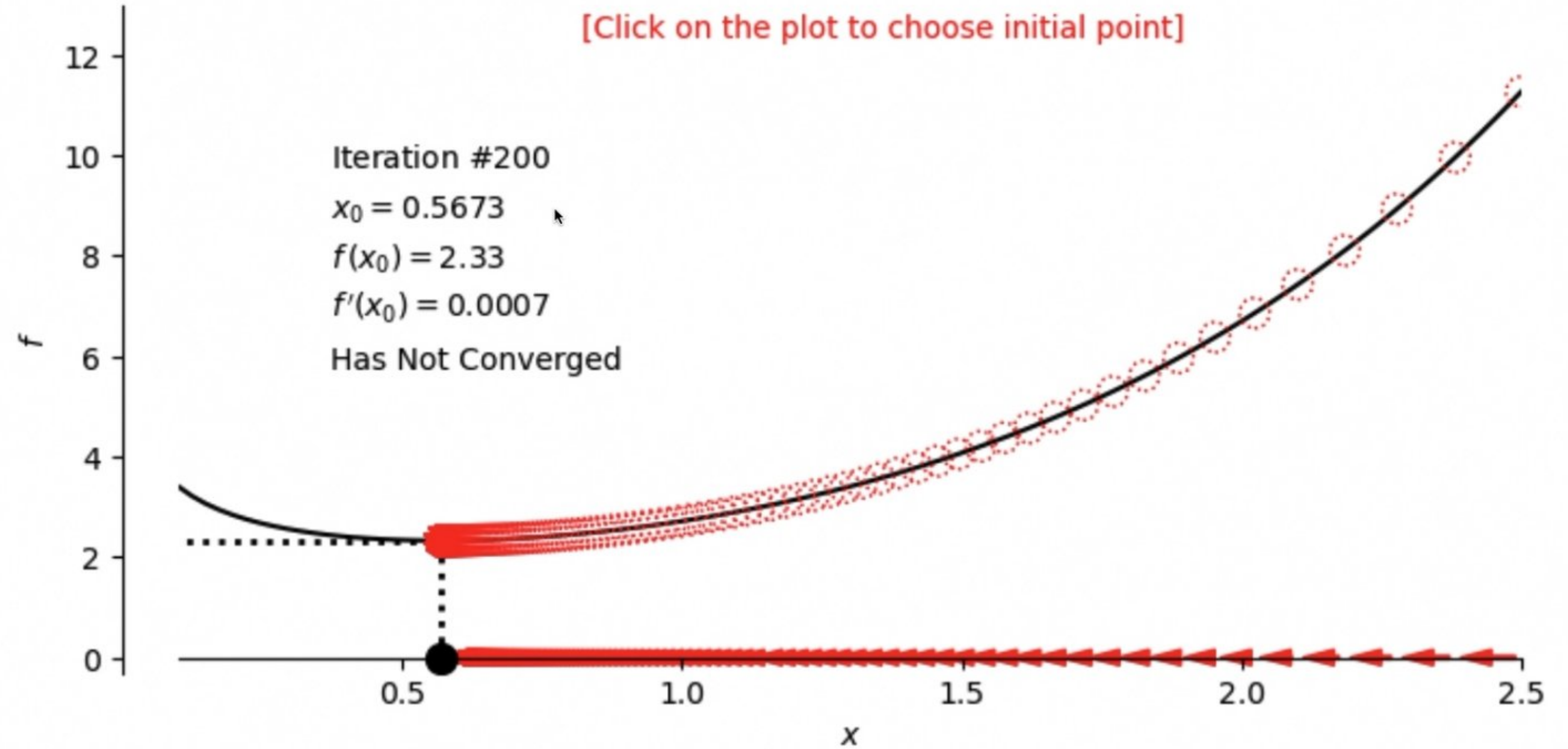
$$\ln(x) + x = 0$$

es:



$$x_{k+1} = x_k - \alpha \cdot f'(x_k),$$

$$x_1 = x_0 - \underbrace{\alpha}_{\text{tasa de aprendizaje}} \frac{df(x_0)}{dx}$$



Funciones con varios mínimos locales

Reg. Linear Simple.

$$\hat{y} = \underbrace{\beta_0}_{2 \text{ param.}} + \underbrace{\beta_1 X}$$

$$Q(\beta_0, \beta_1) = \frac{1}{n} \sum (\hat{y}_i - y_i)^2$$

β_i 's don't depend

$$\frac{\partial Q}{\partial \beta_0} = \frac{\partial Q}{\partial \beta_1} = 0$$

$$= \frac{1}{n} \sum (\underbrace{\beta_0 + \beta_1 X}_i - y_i)^2$$

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \end{bmatrix} = \begin{bmatrix} x_k \\ y_k \end{bmatrix} - \alpha \cdot \nabla f(x_k, y_k),$$

Handwritten annotations for the gradient vector $\nabla f(x_k, y_k)$:

- A circle around $\frac{\partial f(x_k)}{\partial x}$ with an arrow pointing to the x component of the gradient.
- A circle around $\frac{\partial f(y_k)}{\partial y}$ with an arrow pointing to the y component of the gradient.

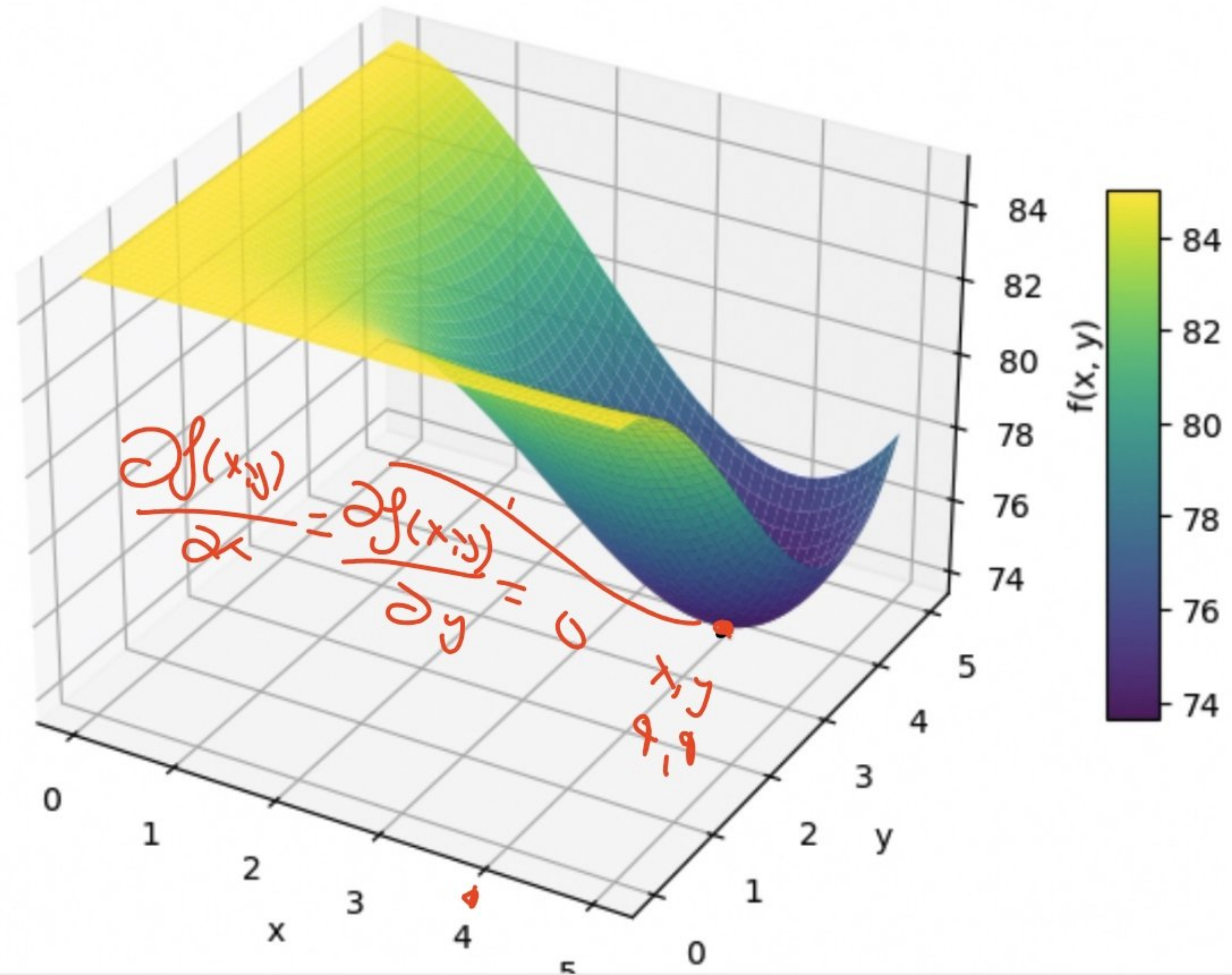
tud del paso en cada iteración.

Handwritten definition of the gradient vector:

$$\nabla = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y} \right)$$

Handwritten update rule for x :

$$x_{k+1} = x_k - \alpha \frac{df'(x_k)}{dx}$$

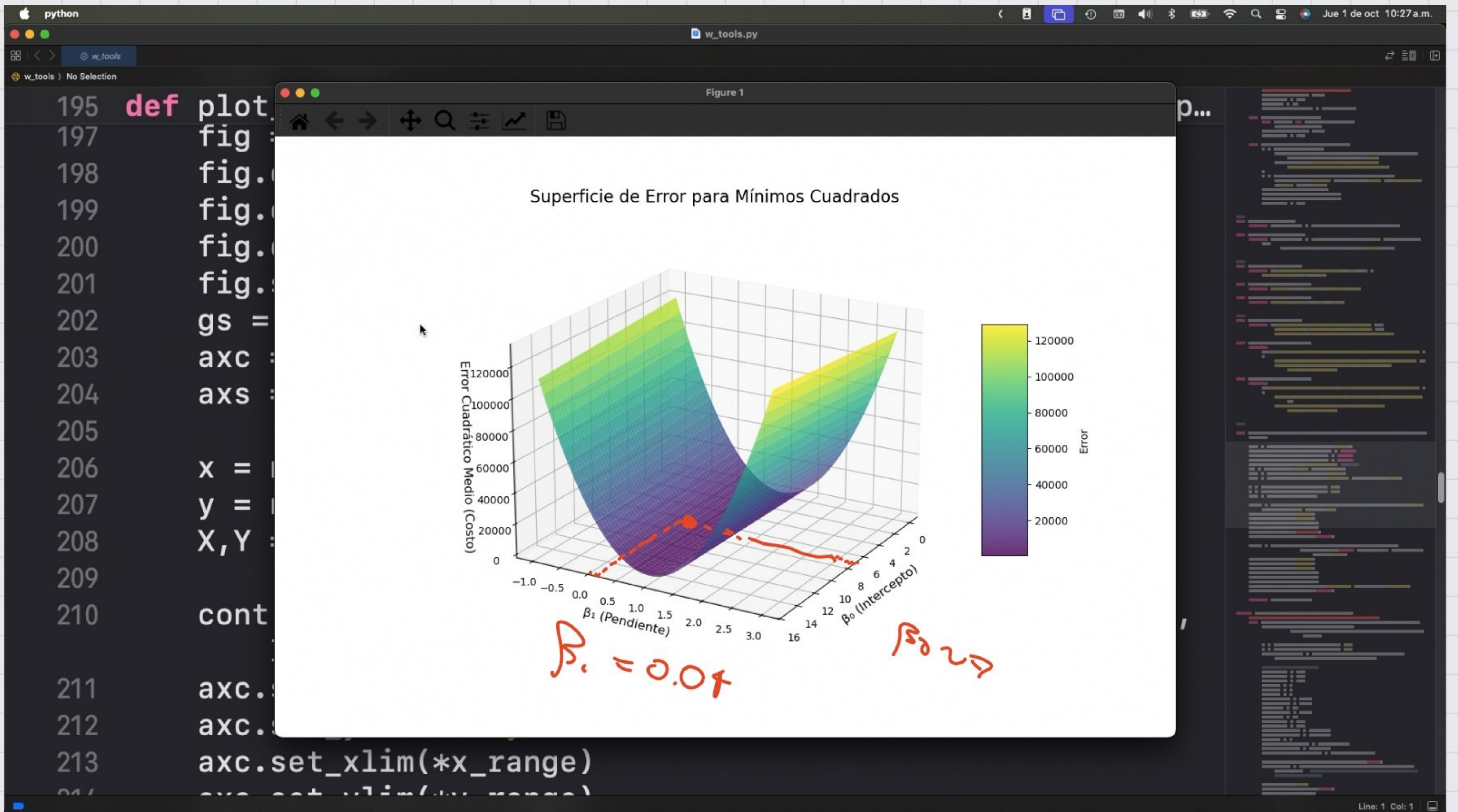


	coef	std err	t	P> t	[0.025
β_0 Intercept	7.0326	0.458	15.360	0.000	6.130
β_1 TV	0.0475	0.003	17.668	0.000	0.042

Omnibus: 0.531 Durbin-Watson: 1.93

Prob(Omnibus): 0.767 Jarque-Bera (JB): 0.66

Skew: -0.089 Prob(JB): 0.71



X

```
[17]: def gradient_descent(dfdx, dfdy, x, y, learning_rate_x = 0.1, learning_rate_y=0.1, num_iterations = 100):
    for iteration in range(num_iterations):
        x, y = x - learning_rate_x * dfdx(x, y), y - learning_rate_y * dfdy(x, y)

        if iteration >= (num_iterations - 5):
            print(f"(x,y):{x,y}")
    return x, y
```

```
[18]: gradient_descent(dfdx=dfdb0, dfdy=dfdb1, x=8.00, y=0.1, learning_rate_x=0.00001, learning_rate_y=0.00001, num_iterations=100)

(x,y):(8.383908217423615, 0.947790236260353)
(x,y):(8.38394524529179, 0.9477900520498101)
(x,y):(8.383982272972037, 0.9477898678402024)
(x,y):(8.38401930046436, 0.9477896836315294)
(x,y):(8.384056327768759, 0.9477894994237913)
[18]: (8.384056327768759, 0.9477894994237913)
```

$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = \begin{pmatrix} x_k \\ y_k \end{pmatrix} - \alpha \begin{pmatrix} \frac{\partial \mathcal{L}(\beta_0^k, \beta_1^k)}{\partial \beta_0} \\ \frac{\partial \mathcal{L}(\beta_0^k, \beta_1^k)}{\partial \beta_1} \end{pmatrix}$$

[]:

[]:

[]:

[]:

[]:

	coef	std err	t	P> t	[0.025	0.
--	------	---------	---	------	--------	----

β_0 **Intercept** -1.388e-16 0.0444 -3.13e-15 1.000 -0.087 (

β_1 **TV** 0.7822 0.044 17.668 0.000 0.695 (

Omnibus:	0.531	Durbin-Watson:	1.935
-----------------	-------	-----------------------	-------

Prob(Omnibus):	0.767	Jarque-Bera (JB):	0.669
-----------------------	-------	--------------------------	-------

Skew:	-0.089	Prob(JB):	0.716
--------------	--------	------------------	-------

Kurtosis:	2.779	Cond. No.	1.00
------------------	-------	------------------	------

```
if iteration >= (num_iterations -5):  
    print(f"(x,y):{x,y}")  
return x, y
```

[78]: gradient_descent(dfdx=dfdb0, dfdy=dfdb1, x=0.1, y=1.0, learning_rate_x=0.01, le

(x,y):(-2.642826921026308e-16, 0.7822244248616093)

(x,y):(-2.642826921026308e-16, 0.7822244248616093)

(x,y):(-2.642826921026308e-16, 0.7822244248616093)

(x,y):(-2.642826921026308e-16, 0.7822244248616093)

(x,y):(-2.642826921026308e-16, 0.7822244248616093)

[78]: (-2.642826921026308e-16, 0.7822244248616093)

[]:

[]:

[]:

Reg. Lined

Python (statsmodels,
scikit-learn)

Analytics

$$\beta = (X^T X)^{-1} X^T y$$

Approx

$$\beta_{k+1} = \beta_k - \eta \nabla J$$

$X \rightarrow$

1	2	3
4	5	6
7	8	9

Flatten $\rightarrow$

x_1	x_2	$\dots$						x_{10}	y
1	2	3	4	5	6	7	8	9	2

$y \rightarrow$